

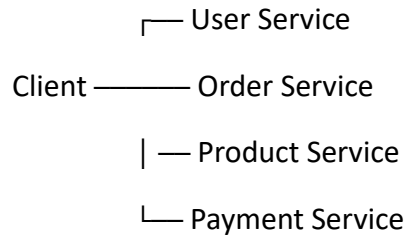
TASK 01 — API Gateway Routing, Filtering & Request Policies

Step 1 — Understand API Gateway

What is an API Gateway?

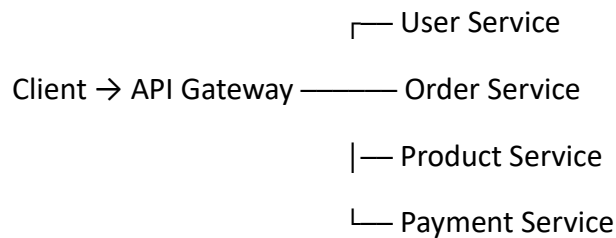
An **API Gateway** is a single entry point between the **client** and your **microservices**.

Without an API Gateway:



The client has to know about every microservice.

With an API Gateway:



The client communicates with **only the Gateway**, and the Gateway forwards the request to the correct service.

2. Why do microservices need a Gateway?

Suppose you have four services:

- User Service
- Order Service
- Product Service
- Payment Service

If the client directly communicates with every service, it needs to know:

- Service IP address
- Service port

- Service API URL

This makes the application difficult to manage.

With an API Gateway:

Client



API Gateway



Correct Microservice

The client only needs to know the Gateway address.

Main advantages

1. Single entry point

The client communicates with one URL.

2. Routing

Gateway sends requests to the correct service.

3. Security

Authentication and authorization can be handled at the Gateway.

4. Logging

Gateway can record incoming requests and responses.

5. Load balancing

Requests can be distributed between multiple instances of a service.

6. Rate limiting

Gateway can restrict excessive requests.

7. Centralized configuration

Common functionality can be managed in one place.

Difference Between Gateway and Individual Services

API Gateway

The Gateway is responsible for **traffic management**.

Routing

Security

Logging

Rate limiting

Correlation ID

Error handling

Individual Microservice

A microservice is responsible for **business functionality**.

For example:

User Service

Create User

Get User

Update User

Delete User

Order Service

Create Order

Get Order

Cancel Order

Product Service

Create Product

Get Product

Update Product

Delete Product

Payment Service

Create Payment

Payment Status

Refund

Gateway vs Service — Simple Table

Feature	API Gateway	Microservice
Main purpose	Traffic management	Business logic
Routing	Yes	No
Authentication	Can handle	Can also validate
Logging	Centralized	Service-specific
Correlation ID	Can generate	Can propagate
Product logic	No	Yes
Order logic	No	Yes
Payment logic	No	Yes
Database	Usually no business DB	Usually owns data
Deployment	Separate application	Separate application

Client-to-Gateway Communication

The **client sends an HTTP request to the Gateway**.

For example:

GET http://localhost:8080/api/products/10

The client is not directly contacting Product Service.

The communication is:

Client

| HTTP Request

API Gateway

Gateway-to-Service Communication

After receiving the request, the Gateway determines where it should go.

Suppose:

GET /api/products/10

The Gateway sees:

/api/products/**

and routes it to:

Product Service :8083

So:

Client



API Gateway :8080



Product Service :8083

The response comes back:

Product Service



API Gateway



Client

STEP 2 — Review Existing Services

Let's assume your ports are:

User Service → 8081

Order Service → 8082

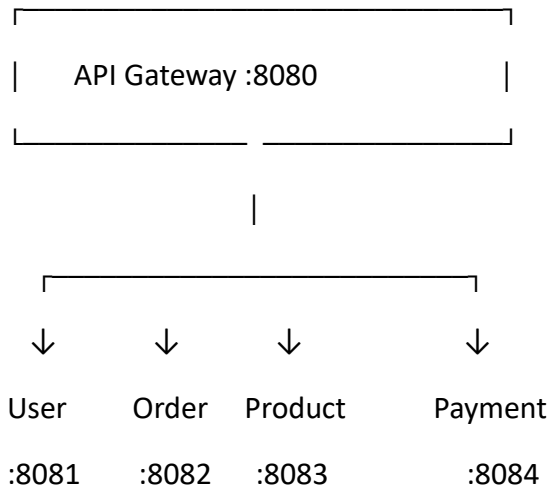
Product Service → 8083

Payment Service → 8084

The new Gateway will run on:

API Gateway → 8080

So:



User Service

The User Service manages user-related functionality.

For example:

GET /users/1

POST /users

PUT /users/1

DELETE /users/1

Example responsibility:

User Service



User Controller



User Service Logic



User Repository



User Database

Order Service

Order Service manages orders.

Examples:

GET /orders/1

POST /orders

PUT /orders/1

DELETE /orders/1

Architecture:

Order Service



Order Controller



Order Business Logic



Order Repository



Order Database

Product Service

Product Service manages products.

Examples:

GET /products/1

POST /products

PUT /products/1

DELETE /products/1

Payment Service

Payment Service manages payments.

Examples:

POST /payments

GET /payments/1

GET /payments/status/1

STEP 3 — Create Spring Cloud Gateway Project

Create a new Spring Boot project using Spring Initializr.

[Spring Initializr](#)

Choose:

Project: Maven

Language: Java

Packaging: Jar

Java: 17

Add:

Spring Cloud Gateway

Spring Boot Actuator

ApiGatewayApplication.java

```
package com.example.apigateway;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication

public class ApiGatewayApplication {

    public static void main(String[] args) {

        SpringApplication.run(
            ApiGatewayApplication.class,
            args
        );
    }
}
```

pom.xml

For a Spring Boot/Spring Cloud combination generated by Initializr, your dependencies should include the Gateway, Actuator, and Resilience4j starter.

Conceptually:

```
<dependencies>

  <!-- Spring Cloud Gateway -->

  <dependency>

    <groupId>org.springframework.cloud</groupId>

    <artifactId>spring-cloud-starter-gateway</artifactId>

  </dependency>

  <!-- Resilience4j Circuit Breaker -->

  <dependency>

    <groupId>org.springframework.cloud</groupId>

    <artifactId>

      spring-cloud-starter-circuitbreaker-reactor-resilience4j

    </artifactId>

  </dependency>

  <!-- Actuator -->

  <dependency>

    <groupId>org.springframework.boot</groupId>

    <artifactId>spring-boot-starter-actuator</artifactId>

  </dependency>

</dependencies>
```

STEP 4 — Configure Service Routes

server:

port: 8080

spring:

application:

name: api-gateway

cloud:

gateway:

routes:

- id: user-service

uri: http://localhost:8081

predicates:

- Path=/api/users/**

- id: order-service

uri: http://localhost:8082

predicates:

- Path=/api/orders/**

- id: product-service

uri: http://localhost:8083

predicates:

- Path=/api/products/**

- id: payment-service

uri: http://localhost:8084

predicates:

- Path=/api/payments/**

Understanding Route Configuration

Consider:

- id: product-service

uri: http://localhost:8083

predicates:

- Path=/api/products/**

There are three important parts.

1. ID

id: product-service

This is the unique name of the route.

2. URI

uri: http://localhost:8083

This is where the request should be sent.

3. Predicate

Path=/api/products/**

This determines when the route should match.

Spring Cloud Gateway supports route predicates and filters through configuration, including the Path predicate used here.

STEP 5 — Test Each Route Using Postman

Start:

User Service → 8081

Order Service → 8082

Product Service → 8083

Payment Service → 8084

API Gateway → 8080

Test 1 — User

GET http://localhost:8080/api/users/1

Flow:

Postman



Gateway :8080

↓

/api/users/**

↓

User Service :8081

Test 2 — Order

GET http://localhost:8080/api/orders/1

Flow:

Postman

↓

Gateway :8080

↓

/api/orders/**

↓

Order Service :8082

Test 3 — Product

GET http://localhost:8080/api/products/1

Flow:

Postman

↓

Gateway :8080

↓

/api/products/**

↓

Product Service :8083

Test 4 — Payment

GET http://localhost:8080/api/payments/1

Flow:

Postman



Gateway :8080



/api/payments/**



Payment Service :8084

STEP 6 — Add Gateway Request Logging

Request ID : 9c123abc

HTTP Method : GET

URL : /api/products/10

Service : product-service

Response Status : 200

Response Time : 124 ms

Why Logging Is Important

Suppose the client says:

Product API is not working.

You can look at the Gateway logs:

Request ID: ABC123

Method: GET

URL: /api/products/10

Service: product-service

Status: 500

Response Time: 2000 ms

Now you know:

Request reached Gateway



Gateway routed to Product Service



Product Service returned 500

This makes troubleshooting much easier.

STEP 7 — Add Correlation / Request ID

Every request should receive a unique ID.

Example:

REQ-123456

or a UUID:

550e8400-e29b-41d4-a716-446655440000

Flow:

Client



Gateway



Generate Request ID



REQ-ABC123

Why Do We Need Request ID

Consider:

Client



Gateway



Order Service



Payment Service



Database

A single user operation may create many log entries.

Without a request ID:

Gateway log

Order log

Payment log

Database log

It can be difficult to know which entries belong to the same request.

With:

X-Correlation-ID: REQ-ABC123

all services can log:

REQ-ABC123

Then you can search the logs for:

REQ-ABC123

and trace the request.

STEP 8 — Forward Correlation ID

The Gateway should forward the same ID to the downstream service.

Example:

X-Correlation-ID: REQ-ABC123

Flow:

Client

|

API Gateway

| X-Correlation-ID: REQ-ABC123

Order Service

| X-Correlation-ID: REQ-ABC123

Payment Service

Global Logging Filter

Create:

GatewayLoggingFilter.java

```
package com.example.apigateway.filter;

import java.util.UUID;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.cloud.gateway.filter.GatewayFilterChain;
import org.springframework.cloud.gateway.filter.GlobalFilter;
import org.springframework.cloud.gateway.support.ServerWebExchangeUtils;
import org.springframework.core.Ordered;
import org.springframework.stereotype.Component;
import org.springframework.web.server.ServerWebExchange;
import reactor.core.publisher.Mono;

@Component
public class GatewayLoggingFilter

    implements GlobalFilter, Ordered {

    private static final Logger log =

        LoggerFactory.getLogger(

            GatewayLoggingFilter.class

        );
```

```

private static final String CORRELATION_ID =

    "X-Correlation-ID";

@Override

public Mono<Void> filter(

    ServerWebExchange exchange,

    GatewayFilterChain chain) {

String requestId =

    UUID.randomUUID().toString();

long startTime =

    System.currentTimeMillis();

String method =

    exchange.getRequest()

        .getMethod()

        .name();

String url =

    exchange.getRequest()

        .getURI()

        .getPath();

ServerWebExchange modifiedExchange =

    exchange.mutate()

        .request(request ->

            request.headers(headers ->

                headers.set(

                    CORRELATION_ID,

                    requestId

                )

            )

        )

```

```

        )
    )
    .build();
return chain
    .filter(modifiedExchange)
    .doFinally(signal -> {
        long responseTime =
            System.currentTimeMillis()
            - startTime;
        Integer status = 0;
        if (modifiedExchange
            .getResponse()
            .getStatusCode() != null) {
            status =
                modifiedExchange
                    .getResponse()
                    .getStatusCode()
                    .value();
        }
        String routeId =
            modifiedExchange
                .getAttribute(
                    ServerWebExchangeUtils
                        .GATEWAY_ROUTE_ATTR
                );
        log.info(

```

```

        "Request ID: {} | " +
        "Method: {} | " +
        "URL: {} | " +
        "Service: {} | " +
        "Status: {} | " +
        "Response Time: {} ms",
        requestId,
        method,
        url,
        routeId,
        status,
        responseTime
    );
});
}

@Override
public int getOrder() {
    return Ordered.HIGHEST_PRECEDENCE;
}
}

```

GlobalFilter is appropriate here because it can be applied across routes, and filter ordering controls when its pre/post logic runs.

What Happens When a Request Arrives

Suppose:

GET /api/products/10

1. Gateway receives request

GET /api/products/10

2. Generate ID

REQ-ABC123

3. Add header

X-Correlation-ID: REQ-ABC123

4. Find route

/api/products/**

5. Forward

Product Service :8083

6. Receive response

200 OK

7. Calculate response time

125 ms

8. Log everything

Request ID: REQ-ABC123

Method: GET

URL: /api/products/10

Service: product-service

Status: 200

Response Time: 125 ms

STEP 9 — Test Service Unavailable Scenario

Start all services.

Then stop:

Product Service

So: Gateway :8080 → RUNNING

Product :8083 → STOPPED

Now call:

GET <http://localhost:8080/api/products/1>

The Gateway tries:

Gateway

↓

Product Service

X

Service unavailable

Instead of allowing an uncontrolled error, we want:

HTTP 503

with a meaningful response.

Circuit Breaker

A **Circuit Breaker** prevents the Gateway from repeatedly calling a service that is failing.

Basic idea:

Normal:

Gateway

↓

Product Service

↓

200 OK

Failure:

Gateway

↓

Product Service

X

Failure

After enough failures:

Gateway



Circuit Breaker



Fallback

Circuit Breaker States

CLOSED

Everything is normal.

Gateway → Service → Success

OPEN

Service is failing.

Gateway



Circuit Breaker



Fallback

The failing service is temporarily avoided.

HALF-OPEN

After a waiting period, the circuit breaker allows a test request.

Circuit Breaker



Test request



Service

/ \

Success Failure

↓ ↓

Closed Open

Add Resilience4j Configuration

For the common Spring Cloud configuration style:

resilience4j:

 circuitbreaker:

 instances:

 userCircuitBreaker:

 slidingWindowSize: 10

 failureRateThreshold: 50

 waitDurationInOpenState: 10s

 orderCircuitBreaker:

 slidingWindowSize: 10

 failureRateThreshold: 50

 waitDurationInOpenState: 10s

 productCircuitBreaker:

 slidingWindowSize: 10

 failureRateThreshold: 50

 waitDurationInOpenState: 10s

 paymentCircuitBreaker:

 slidingWindowSize: 10

 failureRateThreshold: 50

 waitDurationInOpenState: 10s

For example:

10 requests

↓

5 or more failures

↓

50% failure threshold

↓

Circuit can open

Add Circuit Breaker to Routes

For the compatible Gateway/Resilience4j version, the route can be configured like:

spring:

cloud:

gateway:

routes:

- id: product-service

uri: http://localhost:8083

predicates:

- Path=/api/products/**

filters:

- name: CircuitBreaker

args:

name: productCircuitBreaker

fallbackUri: forward:/fallback/product

Do the same for the other services.

Create Fallback Controller

Create:

controller/GatewayFallbackController.java

package com.example.apigateway.controller;

```
import java.util.HashMap;

import java.util.Map;

import org.springframework.http.HttpStatus;

import org.springframework.http.ResponseEntity;

import org.springframework.web.bind.annotation.GetMapping;

import org.springframework.web.bind.annotation.RestController;

@RestController

public class GatewayFallbackController {

    @GetMapping("/fallback/user")

    public ResponseEntity<Map<String, Object>>

    userFallback() {

        return fallback(

            "User Service is currently unavailable"

        );

    }

    @GetMapping("/fallback/order")

    public ResponseEntity<Map<String, Object>>

    orderFallback() {

        return fallback(

            "Order Service is currently unavailable"

        );

    }

    @GetMapping("/fallback/product")

    public ResponseEntity<Map<String, Object>>

    productFallback() {

        return fallback(
```

```

        "Product Service is currently unavailable"
    );
}

@GetMapping("/fallback/payment")
public ResponseEntity<Map<String, Object>>
paymentFallback() {
    return fallback(
        "Payment Service is currently unavailable"
    );
}

private ResponseEntity<Map<String, Object>>
fallback(String message) {
    Map<String, Object> response =
        new HashMap<>();
    response.put("status", 503);
    response.put("message", message);
    response.put(
        "timestamp",
        System.currentTimeMillis()
    );
    return ResponseEntity
        .status(
            HttpStatus.SERVICE_UNAVAILABLE
        )
        .body(response);
}
}

```

Test Step 9

Stop:

Product Service

Then:

GET <http://localhost:8080/api/products/1>

Expected:

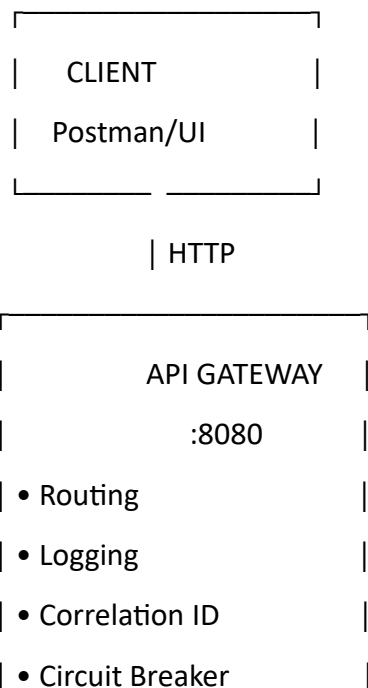
HTTP/1.1 503 Service Unavailable

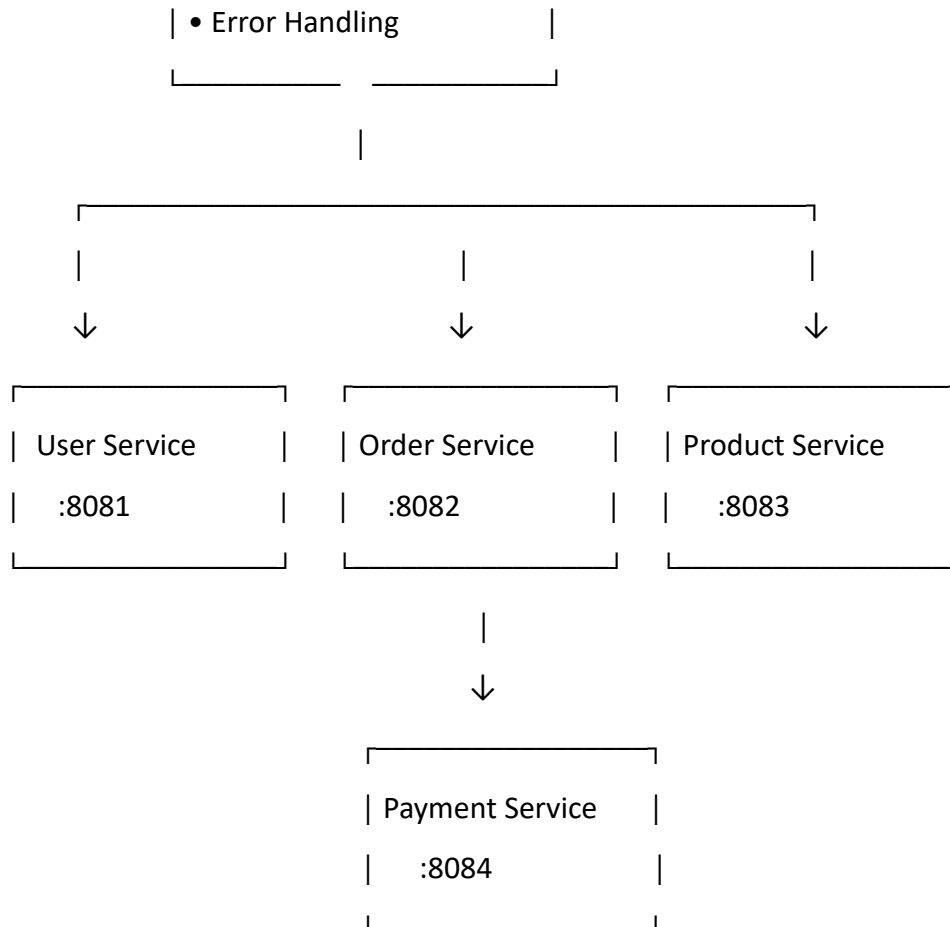
Response:

```
{  
  "status": 503,  
  "message": "Product Service is currently unavailable",  
  "timestamp": 1791190000000  
}
```

This is called a **controlled response**.

STEP 10 — Create Architecture Diagram





Complete Request Example

GET <http://localhost:8080/api/products/10>

Step 1 — Client sends request

Client



GET [/api/products/10](#)

Step 2 — Gateway receives request

API Gateway :8080

Gateway checks the route.

[/api/products/**](#)

matches:

Product Service

Step 3 — Generate correlation ID

Gateway creates:

REQ-ABC123

Step 4 — Add correlation header

X-Correlation-ID: REQ-ABC123

Step 5 — Forward request

Gateway :8080



Product Service :8083

Step 6 — Product Service processes request

For example:

Product ID = 10

Product Service returns:

200 OK

Step 7 — Gateway logs

Request ID: REQ-ABC123

Method: GET

URL: /api/products/10

Service: product-service

Status: 200

Response Time: 120 ms

Step 8 — Response goes back

Product Service



API Gateway



Client